



Manel Ocana

BLOCK 3 - FRAMEWORK

INDEX

- Introduction
- Philosophie générale du projet
- Présentation
- Structure modulaire du projet
- Flux de création de l'application
- Templates
- Routes (Blueprints)
- Connexion à la base de données
- Modèles (SQLAlchemy)
- Extensions
- Initialisation de l'application
- Lancement de l'application
- Authentification
- Gestion des rôles
- Interface d'administration (CRUD)
- Formulaires — WTForms
- Gestion des images
- Le flux de gestion des images
- Conclusion

INTRODUCTION

Une application web monolithique, développée en Python avec Flask, connectée à une base de données MySQL.

L'objectif du site est de présenter du contenu visuel et informatif.

L'application dispose d'un système d'authentification avec gestion des rôles, ainsi que d'un panneau d'administration, avec une architecture modulaire.

Elle respecte les bonnes pratiques en matière de sécurité et de SEO.

PHILOSOPHIE GÉNÉRALE DU PROJET

- Application web développée avec Flask
- Frontend classique : HTML + CSS
- Backend avec Flask + SQLAlchemy
- Base de données MySQL
- Portfolio et blog dynamiques
- Interface d'administration protégée (authentification + rôles)
- Upload d'images
- Application monolithique (sans React, sans API)

HOME ABOUT

WE MAKE CREATIVE THINGS EVERYDAY

We are digital agency that helps
immersive and engaging user
experiences that

[VIEW OUR WORKS](#)

PRÉSENTATION

Dans cette présentation, je vais me concentrer sur le fonctionnement global de l'application ainsi que sur le flux de travail.

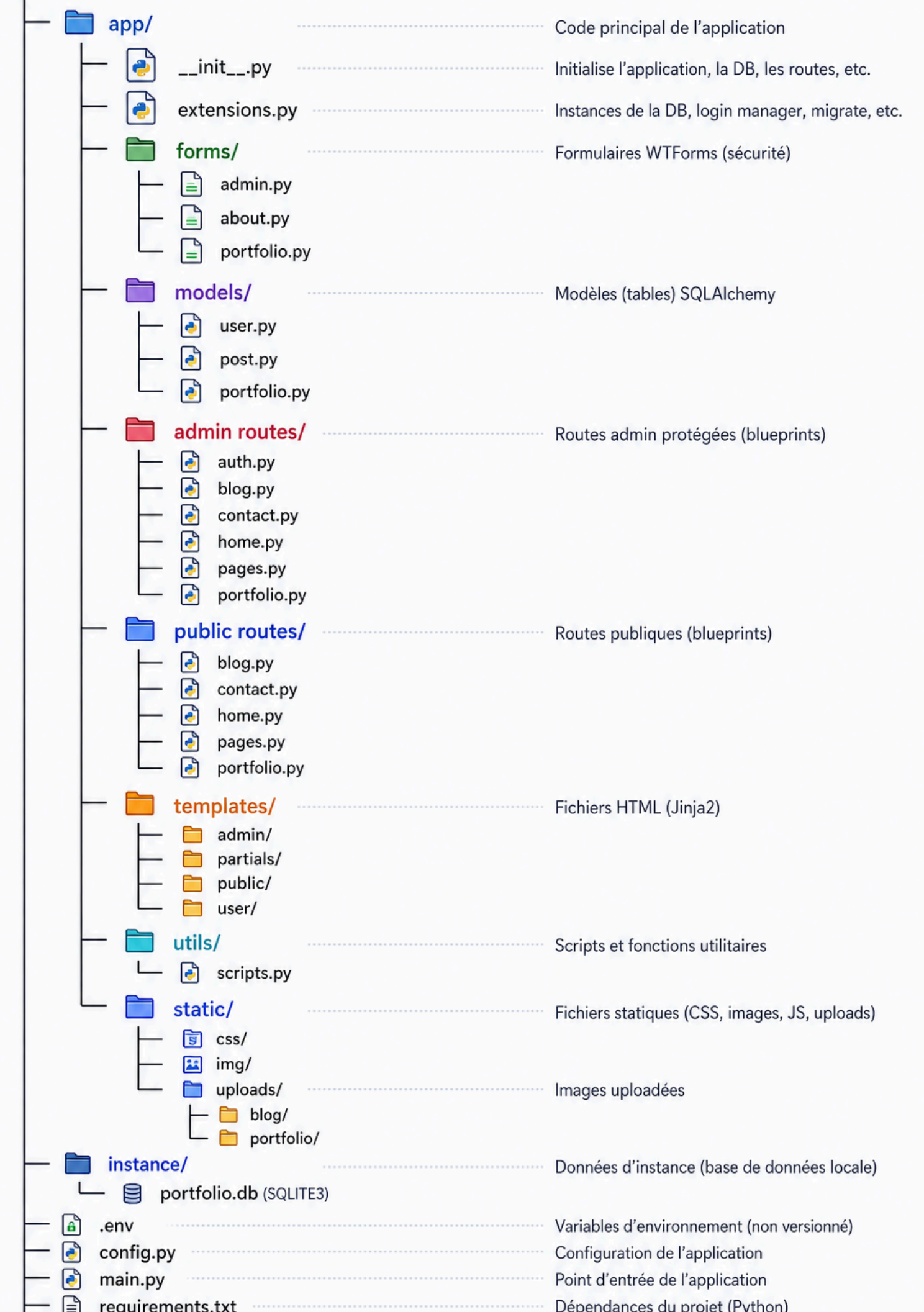
Je vais vous présenter plusieurs extracteurs de code, à titre d'exemples.

Pour plus de détails techniques, ils sont disponibles dans le fichier README.md sur GitHub :

https://github.com/manelocana/Projet_Framework/blob/main/README.md

STRUCTURE MODULAIRE DU PROJET

- `__init__.py` : initialise l'application (app, base de données, routes, etc.)
- `extensions.py` : instances de la base de données, login manager, migrations
- `models/` : modèles (tables MySQL)
- `admin_routes/` : blueprints avec routes protégées
- `public_routes/` : routes publiques
- `templates/` : fichiers HTML
- `forms/` : formulaires sécurisés avec WTForms
- `static/` : fichiers statiques (CSS, images, JS)
- `utils/` : scripts utilitaires (création admin, gestion des images...)



FLUX DE CRÉATION DE L'APPLICATION

RESOLUTION MOBILE

```
@MEDIA(MAX-WIDTH: 1024PX) {  
    CODE CSS  
}
```

RESOLUTION DESKTOP

```
@MEDIA(MIN-WIDTH: 1024PX) {  
    CODE CSS  
}
```

En suivant une approche mobile-first, je développe d'abord la version mobile

puis j'adapte la version desktop avec un decorateur comme ça :

TEMPLATES

Création des templates partiels :

- index.html : structure principale (métadonnées, scripts JS, CSS, header, footer)
- header.html : logo, navigation, login
- footer.html : informations légales

Ensuite, création des pages publiques avec Jinja2 pour le rendu dynamique, pour afficher tout ça qui a dans index, le css, etc...

dans le template:

```
{% extends 'partials/index.html' %}
```

```
{% block extra_css %}
```

```
<link rel="stylesheet" href="{{  
url_for('static', filename='css/  
FICHIER.css') }}">
```

```
{% endblock %}
```

```
{% block content %}
```

ICI LE CONTENU DE LA PAGE

```
{% endblock %}
```


ROUTES (BLUEPRINTS)

Les routes sont organisées via des blueprints afin de séparer clairement la logique, les données et les vues, ce qui facilite la maintenance et le debugging.

- Routes publiques : affichage des données (GET, navigation)
- Routes privées : gestion du CRUD (création, lecture, modification, suppression) et accès à l'administration

Les validations et la gestion des erreurs sont centralisées dans les routes.

Exemple :

```
blog_admin_bp =  
Blueprint('blog_admin', __name__,  
template_folder='../templates',  
url_prefix='/admin')  
  
@blog_admin_bp.route('/blog')  
  
@login_required  
  
@role_required(["admin"])  
  
def blog_list():  
    posts =  
    Post.query.order_by(Post.id.desc()).all()  
  
    return render_template('admin/blog/  
blog_list.html', posts=posts)
```

CONNEXION À LA BASE DE DONNÉES

Configuration via config.py

Avec utilisation de dotenv, pour sécuriser les variables sensibles.

Et les importer depuis .env

```
import os
from dotenv import load_dotenv

load_dotenv()

BASE_DIR = os.path.abspath(os.path.dirname(__file__))

class Config:
    SECRET_KEY = os.environ.get('SECRET_KEY')
    SQLALCHEMY_DATABASE_URI = os.getenv('DATABASE_URL')
    SQLALCHEMY_TRACK_MODIFICATIONS = False
    UPLOAD_FOLDER = os.path.join(BASE_DIR, 'app', 'static', 'uploads')
```

```
from app.extensions import db
```

```
class Post(db.Model):  
    id = db.Column(db.Integer, primary_key=True)  
  
    title = db.Column(db.String(100), nullable=False)  
  
    description = db.Column(db.Text)  
  
    image = db.Column(db.String(200))  
  
    """ relation post => user_id """  
  
    user_id = db.Column(db.Integer,  
db.ForeignKey('user.id'), nullable=False)
```

MODÈLES (SQLALCHEMY)

Création des modèles Python
représentant les tables MySQL,
avec gestion des relations avec
autres tables (clés étrangères)

EXTENSIONS

Initialisation des services principaux :

- Base de données
- Authentification
- Migrations

Exemple :

```
from flask_sqlalchemy import SQLAlchemy
from flask_login import LoginManager
from flask_migrate import Migrate
```

```
db = SQLAlchemy()
login_manager = LoginManager()
migrate = Migrate()
```

INITIALISATION DE L'APPLICATION

Dans le fichier `__init__.py`

Fonction `create_app()` :

- Chargement de la configuration
- Initialisation des extensions
- Enregistrement des blueprints

```
import pymysql
pymysql.install_as_MySQLdb()

def create_app(config_class=Config):

    app = Flask(__name__)

    """ configuration depuis la class Config """
    app.config.from_object(config_class)

    """ initialiser la database et regarder si a des migrations """
    db.init_app(app)
    migrate.init_app(app, db)

    """ initialiser le login """
    login_manager.init_app(app)
    login_manager.login_view = 'auth.login'

    """ registrer les blueprint qui contient les routes """
    app.register_blueprint(home_bp)

    return app
```



```
from app import create_app  
from app.extensions import db
```

```
app = create_app()
```

```
if __name__ == "__main__":  
    app.run(debug=True)
```

LANCEMENT DE L'APPLICATION

Dans le fichier main.py

En production :

- debug=False
- Déploiement avec Gunicorn + Nginx

AUTHENTIFICATION

- Protection des routes avec `login_required`
- Conditions dans les templates avec Jinja2

```
from flask_login import login_required
```

```
@login_required  
def portfolio_new():  
    ...
```

Et dans les templates:

```
{% if current_user.is_authenticated %}  
    <a>Edit</a>  
    <form>Delete</form>  
{% endif %}
```



```

def role_required(roles):
def decorator(f):

    """ wraps pour conserver les metadonnées de la
    function (nom, docstring, metadata ... ) """

    @wraps(f)
    def wrapper(*args, **kwargs):

        """ si l'utilisateur il est pas connecté, error http-401 """
        if not current_user.is_authenticated:
            abort(401) # no log

        """ si le role n'est pas autorisé, return http-403 """
        if current_user.role not in roles:
            abort(403) # no auth

    return f(*args, **kwargs)
    return wrapper
    return decorator

```

GESTION DES RÔLES

Création d'un décorateur personnalisé pour restreindre l'accès :

- 401 : utilisateur non authentifié
- 403 : accès non autorisé

ement

	Title	Description	Actions
	dfghjd	dfghdfg...	EditDelete
	dfghdffhj	dfghdf...	EditDelete
	dfghhdfgh	dfghdfghdf...	EditDelete
			EditDelete

INTERFACE D’ADMINISTRATION (CRUD)

Un panneau d’administration permet de gérer :

- Création
- Lecture
- Modification
- Suppression

Aucune logique métier n’est présente dans les templates publics.

FORMULAIRES – WTFORMS

Les formulaires sont définis avec Flask-WTF, intégrant validation côté serveur et protection CSRF automatique via un token dans chaque formulaire.

Exemples :

- LoginForm — email, mot de passe, “se souvenir de moi”
- RegisterForm — username, email, mot de passe + confirmation
- PostForm — titre, contenu (éditeur rich-text), image
- ProjectForm — titre, description, upload multiple d’images

Dans les templates, Jinja rend automatiquement le token CSRF avec :
`{{ form.hidden_tag() }}`

GESTION DES IMAGES

Dans la partie admin, lors de la création ou modification de contenu, il est possible d'uploader des images.

Formulaire : (important pas oublier, dans le template)

```
<form method="POST" enctype="multipart/form-data">
```

LE FLUX DE GESTION DES IMAGES

SUIT LES ÉTAPES SUIVANTES :

- Le formulaire reçoit le fichier via FileField.
- Validation : extension autorisée, taille maximale et vérification qu'il s'agit bien d'une image.
- Génération d'un nom sécurisé et unique (uuid + extension).
- Stockage dans app/static/uploads/.
- Seul le nom du fichier est stocké en base de données.
- Dans le template :
- `url_for('static', filename='uploads/' + post.image)`

Mesures de sécurité :

- `secure_filename()` — suppression des caractères dangereux
- Validation du type MIME réel (pas seulement l'extension)
- Limite de taille via `MAX_CONTENT_LENGTH`

TESTS DE L'APPLICATION

Des tests automatiques ont été mis en place avec Pytest afin de vérifier le bon fonctionnement de l'application Flask.

Les tests couvrent :

- Les routes principales et les redirections.
- L'authentification et le contrôle d'accès par rôles.
- Les opérations CRUD (création, modification et suppression de posts).
- La validation des formulaires et la sécurité des mots de passe.

```
===== test session starts =====
platform linux -- Python 3.11.2, pytest-9.0.2, pluggy-1.6.0 -- /home/manel/Entornos_virtuales/Proyectos_Web/Projet_Ilaria/Projet_Framework/venv/bin/python3
cachedir: .pytest_cache
rootdir: /home/manel/Entornos_virtuales/Proyectos_Web/Projet_Ilaria/Projet_Framework
collected 10 items

tests/test_models.py::test_create_user PASSED
tests/test_models.py::test_password_is_hashed PASSED
tests/test_posts.py::test_create_post PASSED
tests/test_posts.py::test_normal_user_cannot_create_post PASSED
tests/test_posts.py::test_create_post_empty_title PASSED
tests/test_posts.py::test_edit_post PASSED
tests/test_posts.py::test_delete_post PASSED
tests/test_routes.py::test_home_page PASSED
tests/test_routes.py::test_dashboard_requires_login PASSED
tests/test_routes.py::test_dashboard_logged_in PASSED

===== warnings summary =====
tests/test_posts.py::test_edit_post
tests/test_posts.py::test_delete_post
/home/manel/Entornos_virtuales/Proyectos_Web/Projet_Ilaria/Projet_Framework/venv/lib/python3.11/site-packages/sqlalchemy/orm/query.py:1775:
Warning: The Query.get() method is considered legacy as of the 1.x series of SQLAlchemy and will be removed in a future release. Please use Session.get() instead. (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/2.0/whatsnew)
  rv = self.get(ident)

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 10 passed, 2 warnings in 0.97s =====
```

Des fixtures ont été utilisées pour créer une base de données de test, des utilisateurs et des sessions simulées, garantissant des tests isolés et reproductibles.

Le résultat final est une application stable avec tous les tests validés.

CONCLUSION

Cette architecture offre une base solide, maintenable et sécurisée pour une application web de contenu.

Points clés :

- Modularité grâce aux Blueprints
- Sécurité en couches
- Réutilisation des templates avec Jinja2
- Fiabilité grâce aux tests pytest
- Scalabilité : ajout facile de nouvelles fonctionnalités

MERCI

Manel Ocana